

Content Operations Dashboard Explanation

A detailed guide to the local Next.js mini project in `C:\Users\Mahan Chaturvedi\OneDrive\Documents\react-project`.

1. Project Purpose

This mini project is a compact content management and social publishing dashboard. It demonstrates a practical Pages Router application while showing the migration target for routing logic: native Next.js middleware instead of a custom Express server.

- Use Next.js 15+, React, and TypeScript as the application foundation.
- Keep routing in the Pages Router to match the current architecture.
- Move route middleware concerns into `middleware.ts`.
- Use CSS Modules for scoped styling.
- Use Motion for polished interface animation.
- Use Radix UI and Base UI for accessible headless components.
- Use Recharts to visualize publishing metrics.

2. Technology Stack

Area	Library / Tool	How it is used
Framework	Next.js 15+	Runs the React app, Pages Router routes, API route, and middleware.
Language	TypeScript	Adds type safety for components, data models, config, and API handlers.
UI primitives	Radix UI Tabs	Provides accessible tab behavior for dashboard sections.
Headless components	Base UI Popover	Provides accessible popover behavior without prescribing styles.
Animation	Motion	Animates the hero and workflow cards.
Charts	Recharts	Renders the scheduled-vs-published bar chart.
Styling	CSS Modules	Scopes page styles to avoid global class-name collisions.

3. File Map

File / Folder	Purpose
package.json	Defines scripts and dependencies for Next.js, React, UI libraries, charts, linting, and TypeScript.
pages/index.tsx	Main dashboard page. It renders the hero, metrics, tabs, chart, workflow cards, and routing popover.
pages/_app.tsx	Imports global CSS and renders every Pages Router page.
pages/api/metrics.ts	API route that returns the sample channel and workflow metrics as JSON.
middleware.ts	Native Next.js middleware that handles /go/:workspace and workspace headers.
data/contentMetrics.ts	Shared sample data and TypeScript types for channel metrics and workflow stages.
styles/globals.css	Global reset, color tokens, font stack, and Base UI portal stacking setup.
styles/Home.module.css	Scoped dashboard layout and responsive visual styling.
next.config.ts	Small Next.js configuration with React strict mode enabled.
tsconfig.json	TypeScript compiler options and @/* import alias setup.
eslint.config.mjs	ESLint flat config for Next.js Core Web Vitals and TypeScript rules.

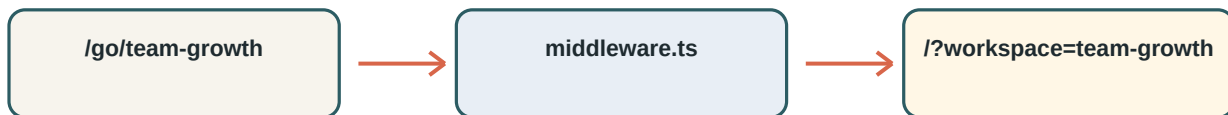
4. Routing Architecture

The application currently uses the Pages Router. The route structure is simple: `pages/index.tsx` maps to the homepage, and `pages/api/metrics.ts` maps to an API endpoint.

Pages Router routes

```
/ -> pages/index.tsx
/api/metrics -> pages/api/metrics.ts
/go/team-growth -> handled by middleware.ts, then rewritten to /?workspace=team-growth
```

Middleware flow



The browser URL stays friendly while Next middleware maps it to the dashboard route.

The old Express idea was a custom server route like `/go/:workspace`. That has now been replaced with native Next.js middleware. When someone visits `/go/team-growth`, middleware extracts `team-growth`, rewrites the request to the dashboard route, and adds the `workspace` value as a query parameter.

Core middleware behavior

1. Match paths for `/` and `/go/:workspace`.
2. If the URL is `/go/:workspace`, clone the request URL.
3. Change the pathname to `/`.
4. Set `workspace` in the query string.
5. Rewrite the request with `NextResponse.rewrite`.
6. Add an `x-content-workspace` response header.

5. Main Dashboard Page

The dashboard in `pages/index.tsx` is intentionally small but complete. It demonstrates how real product surfaces are assembled from data, UI primitives, animation, and charts.

Section	What it demonstrates
Hero	Product-style page heading, short explanation, and Motion entrance animation.
Summary metrics	Derived values calculated from shared data in <code>contentMetrics.ts</code> .
Overview tab	Recharts bar chart comparing scheduled and published content.
Workflow tab	Motion-animated cards showing review, scheduling, and approval counts.
Routing tab	Base UI Popover explaining how the middleware route behaves.

6. Data Model

The sample data lives in `data/contentMetrics.ts`. The `ChannelMetric` type describes each social channel, and the exported arrays are reused by both the page and API route.

```
type ChannelMetric = {  
  channel: string;  
  scheduled: number;  
  published: number;  
  engagement: number;  
};
```

This keeps the mini project easy to understand: there is one source of truth for the dashboard data, and the API route simply exposes that same data as JSON.

7. Styling and Design

- `globals.css` defines global design tokens like background, ink, muted text, teal, gold, and coral.
- `Home.module.css` owns the dashboard layout, card styling, tabs, chart frame, popover, and mobile breakpoints.
- CSS Modules keep styles local to the page component, which is useful in a growing Pages Router codebase.
- Base UI portal guidance is represented by isolating the Next root, so popovers layer predictably above page content.

8. Accessibility Notes

Radix UI and Base UI are used because they provide accessible behavior while letting the project keep its own visual design. The dashboard uses semantic headings, button-based tab triggers, an aria-label for the tab list, and headless popover behavior.

Component	Accessibility benefit
Radix Tabs	Keyboard-friendly tab navigation and proper tab state management.
Base UI Popover	Accessible trigger and popup behavior without custom focus logic.
Semantic HTML	Sections, headings, articles, and main landmarks improve screen-reader structure.
Responsive CSS	Tabs and layout collapse cleanly on narrower screens.

9. How to Run the Project

Open a terminal at the project folder:

```
C:\Users\Mahan Chaturvedi\OneDrive\Documents\react-project
```

Install dependencies and start the dev server:

```
npm install
npm run dev
```

Then open `http://localhost:3000`. To test the middleware route, open `http://localhost:3000/go/team-growth`.

10. What to Explain in an Interview or Review

- The project mirrors a realistic migration path: keep Pages Router pages while moving route middleware out of Express.
- The middleware rewrite preserves a friendly URL pattern while rendering the existing dashboard page.
- Data is centralized in a typed module and reused by both the UI and API route.
- UI behavior comes from accessible primitives instead of hand-rolled tab or popover logic.
- Charts are dynamically imported because browser-oriented chart libraries are safer when rendered client-side.
- CSS Modules keep the styling approach close to the existing stack and avoid global leakage.

11. Suggested Next Improvements

Improvement	Why it matters
Read workspace query in UI	Display the active workspace name when visiting <code>/go/:workspace</code> .
Add tests	Use unit tests for middleware matching and component tests for tabs/popover behavior.
Fetch API metrics	Move the dashboard from imported data to client/server fetching through <code>/api/metrics</code> .

Improvement	Why it matters
Add App Router route	Create an app/ version of the page as a migration exercise.
Persist data	Replace sample data with a small database or JSON-backed service.

Summary

This project is small on purpose, but it touches the important parts of the requested stack: Next.js Pages Router, native middleware, TypeScript data modeling, accessible component primitives, CSS Modules, Motion animations, and Recharts visualization.